Random Testing II

Program Equivalence

CMSC 323: Design and Implementation of Programming Languages

This lecture note is adapted from content created by Robby Findler (Northwestern University) and Vincent St-Amour (Northwestern University) and Shriram Krishnamurthi in Programming Languages: Application and Interpretation.

Interpreter Properties

- Interpreter does not crash
- Produces same result as another implementation (e.g., DrRacket)
- Type checker accurately predicts results (later)
- Program equivalences hold

Program Equivalence Example

For example, we should be able to replace any subexpression with a new variable.

```
{+ 1 2} => {with {x 2}
             {+ 1 x}}
```

Program equivalence is the problem of proving that two programs are equal under some definition of equivalence, e.g., input-output equivalence. The field draws researchers from formal verification, semantics and logic.

Another Example

```
{with {x {+ 5 26}}
      {- x 4}}
```

=>

```
{with {z {+ 5 26}}
      {with {x z}
            {- x 4}}}
```

Examples

```
{+ {+ 3 2}
      5}
```

- replace {+ 3 2} in the expression above with a bound occurrence of an id
- replace 3 in the expression above with a bound occurrence of an id
- replace 2 in the expression above with a bound occurrence of an id

Transformation Strategy

- Given a WAE expression
- Generate a random path for the WAE expression tree.
- Pick a subexpression based on the path to bind to a new id.
- Replace the subexpression with a bound occurrence of the id.

A bit more detail

- Start with a WAE, which is a tree (a leaf is a tree)
- Generate a random path traversing the WAE expression tree.
  - It need not reach a leaf
  - And some rules apply
- Where the path stops, consider that node(itself a WAE) as the root of a subtree (all the way down)
  - The subtree represents a subexpression (i.e., the expression represented by its root WAE)

A bit more detail

- Replace the subexpression, in the original tree, with (var id) for some id
- Bind that id by using a with statement to enclose the entire original WAE.

Example:

- Original WAE:

```
(plus
  (with 'x
        (sub (num 10)
             (plus (num 2) (num 7)))
        (plus (var 'x) (num 3)))
  (sub
    (plus (num 4) (var 'y))
    (with 'z (num 8)
      (sub (var 'z) (num 1)))))
```

- Subtree:

```
(with 'z (num 8)
  (sub (var 'z) (num 1)))
```

Example:

- Replace the subtree with (var 'eq)

```
(plus
  (with 'x
        (sub (num 10)
             (plus (num 2) (num 7)))
        (plus (var 'x) (num 3)))
  (sub
    (plus (num 4) (var 'y))
    (with 'z (num 8)
      (sub (var 'z) (num 1)))))
```

```
(plus
  (with 'x
        (sub (num 10)
             (plus (num 2) (num 7)))
        (plus (var 'x) (num 3)))
  (sub
    (plus (num 4) (var 'y))
    (var 'eq)))
```

Example:

- Now “wrap” the resulting WAE in a **`with`** that assigned to ‘eq the replaced subexpression

```
(plus
  (with 'x
        (sub (num 10)
             (plus (num 2) (num 7)))
        (plus (var 'x) (num 3)))
  (sub
    (plus (num 4) (var 'y))
    (var 'eq)))
```

```
(with 'eq
  (with 'z (num 8)
    (sub (var 'z) (num 1)))
  (plus
    (with 'x
          (sub (num 10)
               (plus (num 2) (num 7)))
          (plus (var 'x) (num 3)))
    (sub
      (plus (num 4) (var 'y))
      (var 'eq))))
```

But of course, there are caveats

- Generate a random path for the WAE expression tree.
  - We have to be careful how we do this (or we could create free ids where there were none before)
- Pick a subexpression based on the path to bind to a new id.
  - We need to know how to generate a random path, etc.

The free ID issue

- Suppose this is our WAE

```
(with 'x
      (plus (num 10) (num 1))
      (plus (var 'x)
            (with 'y (var 'x) (num 3))))
```

- And this is the chosen subexpression

```
(with 'x
      (plus (num 10) (num 1))
      (plus (var 'x)
            (with 'y (var 'x) (num 3))))
```

The free ID issue

- Suppose this is our WAE

```
(with 'x
      (plus (num 10) (num 1))
      (plus (var 'x)
            (with 'y (var 'x) (num 3))))
```

- We replace (var ‘x) with (var ‘eq) and “wrap” the resulting WAE

```
(with 'eq (var 'x)
  (with 'x
        (plus (num 10) (num 1))
        (plus (var 'eq)
              (with 'y (var 'x) (num 3)))))
```

The free ID issue

- Do you see the issue?
- How can we avoid this?

```
(with 'x
      (plus (num 10) (num 1))
      (plus (var 'x)
            (with 'y (var 'x) (num 3))))
```

```
(with 'eq (var 'x)
  (with 'x
        (plus (num 10) (num 1))
        (plus (var 'eq)
              (with 'y (var 'x) (num 3)))))
```

Generating Random Paths

Generate a random path for a WAE expression tree.

- Start at the root of the expression tree
- Flip a coin to determine whether we move further down the tree (i.e., left or right subtree)
- If we reach a leaf node, there is no choice to make. We simply stop generating the path.
- Since we are "lifting" a subexpression out of its original context, we only pick expressions which will not contain free id's
- Always pick the assigned expression of a with (e.g.

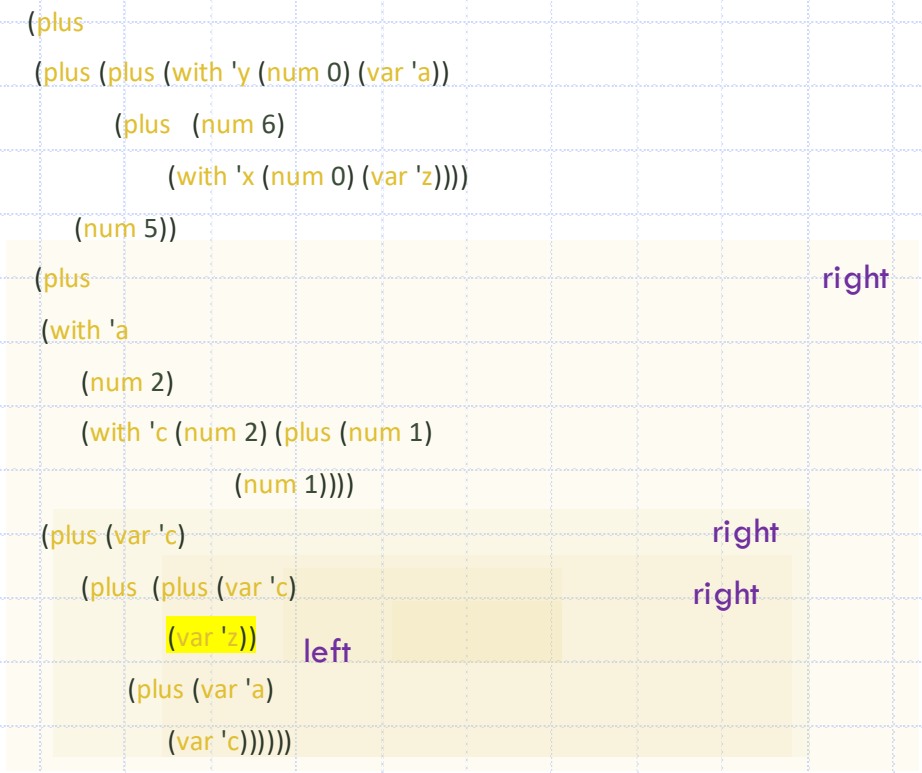
(with 'x (num 3) (plus (var 'x) (var 'x))))

Generating a random path

Generate a random path for a WAE expression tree.

We generate a list of symbols representing the path to the sub expression to be replaced with a bound id. For example,

'(right right right left right)



Generating Random Paths

Generate a random path for a WAE expression tree.

Leaf nodes are chosen by default

```
(generate-random-path : (WAE -> (Listof Symbol)))
(define (generate-random-path wae)
 (type-case WAE wae
  [(num n) empty]
  [(var id) empty]
  [(with id named-expr body) (cons 'left (generate-random-path named-expr))]
                . . .))
```

Generating Random Paths

Generate a random path for a WAE expression tree.

Always pick the named expression of the "with" expressions

```
(generate-random-path : (WAE -> (Listof Symbol)))
(define (generate-random-path wae)
  (type-case WAE wae
    [(num n) empty]
    [(var id) empty]
    [(with id named-expr body) (cons 'left (generate-random-path named-expr))]
. . .))
```

We've been over why this is the case.

Generating Random Paths

Generate a random path for a WAE expression tree.

Flip a coin to determine whether we

move further down the tree

```
;;generate a random-path
(define (get-path wae)
  (type-case WAE wae
    [(num n) '()]
    [(var id) '()]
    [(plus lhs rhs) (if (<= (random 101) 20)
                        '()
                        (case (random 2)
                          [(0) (cons 'left (get-path lhs))]
                          [(1) (cons 'right (get-path rhs))]))]
```

Exercise 1

Given the following paths and WAE expressions, what branch of the expression tree is returned if the path is taken?

Path: '()

WAE: (num 9)

Exercise 2

Given the following paths and WAE expressions, what branch of the expression tree is returned if the path is taken?

Path: '(left right)

WAE: (with 'y (sub (num 4) (num 2)) (var ‘y))

Exercise 3

Given the following paths and WAE expressions, what branch of the expression tree is returned if the path is taken?

Path: '(left left)

WAE: (sub (with 'y (num 9) (var 'y)) (plus (num 7) (num 4)))

Extracting the Subexpression in the Generated Path

If the path is empty, return the original wae

```
;; get sub-expression at the end of the path
(get-sub-exp : ((Listof Symbol) WAE -> WAE))
(define (get-sub-exp path wae)
  (if (empty? path)
      wae
```

Extracting the Subexpression in the Generated Path

For numbers and vars, there are no more paths to take. Return the wae

```
;; get sub-expression at the end of the path
(get-sub-exp : ((Listof Symbol) WAE -> WAE))
(define (get-sub-exp path wae)
  (if (empty? path)
      wae
```

Extracting the Subexpression in the Generated Path

For numbers and vars, there are no more paths to take. Return the wae

```
;; get sub-expression at the end of the path
(get-sub-exp : ((Listof Symbol) WAE -> WAE))
(define (get-sub-exp path wae)
  (if (empty? path)
      wae
      (type-case WAE wae
        [(num n) wae]
        [(var id) wae]
```

Extracting the Subexpression in the Generated Path

For “withs”, we always select the named-expr

```
;; get sub-expression at the end of the path
(get-sub-exp : ((Listof Symbol) WAE -> WAE))
(define (get-sub-exp path wae)
  (if (empty? path)
      wae
      (type-case WAE wae
        [(num n) wae]
        [(var id) wae]
```

Extracting the Subexpression in the Generated Path

For "withs", we always select the named-expr

```
;; get sub-expression at the end of the path
(get-sub-exp : ((Listof Symbol) WAE -> WAE))
(define (get-sub-exp path wae)
  (if (empty? path)
      wae
      (type-case WAE wae
        [(num n) wae]
        [(var id) wae]
        [(with id named-exp body) (get-sub-exp (rest path) named-exp)]
```

Extracting the Subexpression in the Generated Path

For plus and sub, get the lhs or rhs based on whether

the symbol in the path is left or right

```
;; get sub-expression at the end of the path
(get-sub-exp : ((Listof Symbol) WAE -> WAE))
(define (get-sub-exp path wae)
  (if (empty? path)
      wae
      (type-case WAE wae
        [(num n) wae]
        [(var id) wae]
        [(with id named-exp body) (get-sub-exp (rest path) named-exp)]
```

Extracting the Subexpression in the Generated Path

For plus and sub, get the lhs or rhs based on whether the symbol in the path is left or right

```
;; get sub-expression at the end of the path
(get-sub-exp : ((Listof Symbol) WAE -> WAE))
(define (get-sub-exp path wae)
  (if (empty? path)
      wae
      (type-case WAE wae
        [(num n) wae]
        [(var id) wae]
        [(with id named-exp body) (get-sub-exp (rest path) named-exp)]
        [(plus lhs rhs) (case (first path)
                          [(left) (get-sub-exp (rest path) lhs)]
                          [(right) (get-sub-exp (rest path) rhs)])]
        [(sub lhs rhs) (case (first path)
                         [(left) (get-sub-exp (rest path) lhs)]
                         [(right) (get-sub-exp (rest path) rhs)])])))
```

Replacing the Subexpression with a New ID

If control has reached the end of the path
replace the whole expression with the new symbol

```
;; replace the sub-expression with a variable
(replace-sub-exp : ((Listof Symbol) WAE Symbol -> WAE))
(define (replace-sub-exp path wae id)
  (if (empty? path)
      (var id)
```

Replacing the Subexpression with a New ID

Else get the next path branch (a left or a right)

Replace the named expression in a **`with`**

```
;; replace the sub-expression with a variable
(replace-sub-exp : ((Listof Symbol) WAE Symbol -> WAE))
(define (replace-sub-exp path wae id)
  (if (empty? path)
      (var id)
```

Replacing the Subexpression with a New ID

Else get the next path branch (a left or a right)

Replace the named expression in a with

```
;; replace the sub-expression with a variable
(replace-sub-exp : ((Listof Symbol) WAE Symbol -> WAE))
(define (replace-sub-exp path wae id)
  (if (empty? path)
      (var id)
      (type-case WAE wae
        [(with with-id named-exp body) (with with-id (replace-sub-exp (rest path) named-exp id) body)]
```

Replacing the Subexpression with a New ID

In a plus or sub invoke replace-subexpr on lhs or rhs depending on the path symbol

```
;; replace the sub-expression with a variable
(replace-sub-exp : ((Listof Symbol) WAE Symbol -> WAE))
(define (replace-sub-exp path wae id)
  (if (empty? path)
      (var id)
      (type-case WAE wae
        [(with with-id named-exp body) (with with-id (replace-sub-exp (rest path) named-exp id) body)]
```

Replacing the Subexpression with a New ID

In a plus or sub invoke replace-subexpr on lhs or rhs depending on the path symbol

```
;; replace the sub-expression with a variable
(replace-sub-exp : ((Listof Symbol) WAE Symbol -> WAE))
(define (replace-sub-exp path wae id)
  (if (empty? path)
      (var id)
      (type-case WAE wae
        [(with with-id named-exp body) (with with-id (replace-sub-exp (rest path) named-exp id) body)]
        [(plus lhs rhs) (case (first path)
                          [(left) (plus (replace-sub-exp (rest path) lhs id) rhs)]
                          [(right) (plus lhs (replace-sub-exp (rest path) rhs id))])]
```

Replacing the Subexpression with a New ID

What about a var or num?

```
;; replace the sub-expression with a variable
(replace-sub-exp : ((Listof Symbol) WAE Symbol -> WAE))
(define (replace-sub-exp path wae id)
  (if (empty? path)
      (var id)
      (type-case WAE wae
        [(with with-id named-exp body) (with with-id (replace-sub-exp (rest path) named-exp id) body)]
        [(plus lhs rhs) (case (first path)
                          [(left) (plus (replace-sub-exp (rest path) lhs id) rhs)]
                          [(right) (plus lhs (replace-sub-exp (rest path) rhs id))])]
```

Replacing the Subexpression with a New ID

What about a var or num?

```
;; replace the sub-expression with a variable
(replace-sub-exp : ((Listof Symbol) WAE Symbol -> WAE))
(define (replace-sub-exp path wae id)
  (if (empty? path)
      (var id)
      (type-case WAE wae
        [(with with-id named-exp body) (with with-id (replace-sub-exp (rest path) named-exp id) body)]
        [(plus lhs rhs) (case (first path)
                          [(left) (plus (replace-sub-exp (rest path) lhs id) rhs)]
                          [(right) (plus lhs (replace-sub-exp (rest path) rhs id))])]
        [(sub lhs rhs) (case (first path)
                         [(left) (sub (replace-sub-exp (rest path) lhs id) rhs)]
                         [(right) (sub lhs (replace-sub-exp (rest path) rhs id))])]
        [(num n) (var id)]
        [(var id) (var id)])))
```

Replacing the Subexpression with a New ID

What about a var or num? This is better.

```
;; replace the sub-expression with a variable
(replace-sub-exp : ((Listof Symbol) WAE Symbol -> WAE))
(define (replace-sub-exp path wae id)
  (if (empty? path)
      (var id)
      (type-case WAE wae
        [(with with-id named-exp body) (with with-id (replace-sub-exp (rest path) named-exp id) body)]
        [(plus lhs rhs) (case (first path)
                          [(left) (plus (replace-sub-exp (rest path) lhs id) rhs)]
                          [(right) (plus lhs (replace-sub-exp (rest path) rhs id))])]
        [(sub lhs rhs) (case (first path)
                         [(left) (sub (replace-sub-exp (rest path) lhs id) rhs)]
                         [(right) (sub lhs (replace-sub-exp (rest path) rhs id))])]
        [(num n) (var id)]
        [(var ignored) (var id)])))
```

Replacing the Subexpression with a New ID

What about a var or num? Or this.

```
(replace-subexpr : (Symbol (Listof Symbol) WAE -> WAE))
(define (replace-subexpr symbol path wae)
 (if (empty? path) (var symbol)
    (local [(define branch (first path))]
     (type-case WAE wae
       [(with id named-expr body) (with id (replace-subexpr symbol (rest path) named-expr) body)]
       [(plus lhs rhs) (case branch
                         [(left) (plus (replace-subexpr symbol (rest path) lhs) rhs)]
                         [(right) (plus lhs (replace-subexpr symbol (rest path) rhs))])]
       [(sub lhs rhs) (case branch
                        [(left) (sub (replace-subexpr symbol (rest path) lhs) rhs)]
                        [(right) (sub lhs (replace-subexpr symbol (rest path) rhs))])]
       [else (var symbol)])))))
```

Exercise: Implementing the Transformation

Complete make-equivalent-expression

- Generate a random-WAE
- Generate a path from the WAE
- Retrieve the expression at the end of the path
- Replace the expression with a variable
- Enclose the new WAE in a with

Putting it all together

```
;; define a function that does the following
;; 1. receives a wae expression
;; 2. generates a random path to a sub-expression in the wae (path)
;; 3. retrieves the sub-expression at the end of the path (sub-expression)
;; 4. replaces the sub-expression with a random-variable (new-wae)
;; 5. generates and returns an equivalent program with the form (with id sub-expression new-wae)
(define (make-equivalent-expression wae)
  (local [(define path (get-path wae))
          (define sub-exp (get-sub-exp path wae))
          (define id (random-symbol all-symbols))
          (define with-body (replace-sub-exp path wae id))
          (define new-wae (with id sub-exp with-body))]
    (begin
```